Imam Abdulrahman Bin Faisal University
College of Computer Science and Information Technology

ARTI 404 — Natural Language Processing

Lab 5 — Text Representation

Student Name	Ahmed Albuainain
Student ID	2240006128
Section	7MA1

Text Representation

Setup

Run this cell first. It installs the libraries used in this lab and downloads the NLTK tokenizer data.

```
# Install the libraries used in this lab (safe to re-run)
!pip install -q "gensim>=4.3.3" kagglehub

import pandas as pd
import numpy as np
import re
import os
import nltk

nltk.download('punkt')
nltk.download('punkt_tab')

print("Setup complete.")
```

```
'pip' is not recognized as an internal or external command,
operable program or batch file.
```

```
Setup complete.
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\ahmad\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ahmad\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Text representation in NLP means converting text into a numerical form that a machine learning model can understand and process.

What is Embedding?

Embeddings in NLP is a technique where individual words are represented as real-valued vectors and captures inter-word semantics.

In this notebook, We going to intreduce 2 techniques for embedding. These techniques will be used for a machine learning models such as SVM, Random forest, ... ect.



1- TF-IDF



TF-IDF stands for term frequency-inverse document frequency. It is a measure that discounts common words. Used in the fields of information retrieval (IR) and machine learning, that can

quantify the importance of words in a document amongst a collection of documents (also known as a corpus).

Components of TF-IDF

1. **TF (Term Frequency):**
   Measures how frequently a term appears in a document.

$$ \text{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d} $$

2. **IDF (Inverse Document Frequency):**
   Measures how important a term is across all documents. Words that appear in many documents get lower scores.

$$ \text{IDF}(t) = \log \left(\frac{\text{Number of all documents N}}{\text{Number of documents containing the term } t}\right) $$



```
import sys
!{sys.executable} -m pip install gensim
```

```
Requirement already satisfied: gensim in c:\users\ahmad\miniconda3\lib\site-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in c:\users\ahmad\miniconda3\lib\site-packages
(from gensim) (2.3.5)
Requirement already satisfied: scipy>=1.7.0 in c:\users\ahmad\miniconda3\lib\site-packages
(from gensim) (1.17.0)
Requirement already satisfied: smart_open>=1.8.1 in c:\users\ahmad\miniconda3\lib\site-
packages (from gensim) (8.0.1)
Requirement already satisfied: wrapt in c:\users\ahmad\miniconda3\lib\site-packages (from
smart_open>=1.8.1->gensim) (2.4.1)
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
doc_1 = "Data is the oil of the digital economy"
doc_2 = "Data is a new oil"

data = [doc_1, doc_2]
```

```
tfidf = TfidfVectorizer()
result = tfidf.fit_transform(data) # returns sparce matrix
```

```
df = pd.DataFrame(result.toarray(), columns=tfidf.get_feature_names_out())
df
```

	data	digital	economy	is	new	of	oil	the
0	0.243777	0.34262	0.34262	0.243777	0.000000	0.34262	0.243777	0.68524
1	0.448321	0.00000	0.00000	0.448321	0.630099	0.00000	0.448321	0.00000

Cosine similarity

In NLP, Cosine similarity is a metric used to measure how similar the documents are.

$$ \text{cosine similarity} = \frac{A \cdot B}{|A| \times |B|} $$

Where:
$ A \cdot B $ = dot product of vectors A and B
$ |A| $ = magnitude (length) of vector A
$ |B| $ = magnitude (length) of vector B

Intuition

- If vectors point in the **same direction**, cosine similarity = **1** (maximum similarity).
- If vectors are **orthogonal (90° apart)**, cosine similarity = **0** (no similarity).



```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
# Sample text data (replace with your own documents)

doc_1 = "Data is the oil of the digital economy"
doc_2 = "Data is a new oil"

data = [doc_1, doc_2]
```

```
tfidf_vectorizer = TfidfVectorizer() # Create a CountVectorizer instance
vector_matrix = tfidf_vectorizer.fit_transform(data) # Fit and transform the documents
into numerical vectors
```

```
# Calculate the cosine similarity between the documents
cosine_similarity_matrix = cosine_similarity(vector_matrix)

df_cosine = pd.DataFrame(data=cosine_similarity_matrix, index=data, columns=data)

df_cosine
```

	Data is the oil of the digital economy	**Data is a new oil**
Data is the oil of the digital economy	1.000000	0.327871
Data is a new oil	0.327871	1.000000

2- What is Word2vec?

Word2Vec consists of models for generating word embedding. These models are two-layer neural networks having one input layer, one hidden layer, and one output layer.

Word2Vec utilizes two architectures :
1. CBOW (Continuous Bag of Words)
2. **Skip Gram**



Run this command in terminal to install

```
pip install gensim
```

We will use fake and real news dataset to do our expirament. You can find the dataset here: https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset?select=True.csv

```
import pandas as pd
import nltk
import numpy as np
import gensim

from nltk.tokenize import word_tokenize
```

```
# Load the "Fake and real news" dataset (True.csv) from Kaggle
import kagglehub, os

news_path = kagglehub.dataset_download("clmentbisaillon/fake-and-real-news-dataset")
print("Path to dataset files:", news_path)
print("Files:", os.listdir(news_path))

df = pd.read_csv(os.path.join(news_path, "True.csv"),
                 engine='python', on_bad_lines='skip')
print("\nShape:", df.shape)
df.head()
```

```
Downloading to C:\Users\ahmad\.cache\kagglehub\datasets\clmentbisaillon\fake-and-real-news-dataset\1.archive...
```

```
100%|██████████| 41.0M/41.0M [00:04<00:00, 10.3MB/s]
```

```
Extracting files...
```

```
Path to dataset files: C:\Users\ahmad\.cache\kagglehub\datasets\clmentbisaillon\fake-and-real-news-dataset\versions\1
Files: ['Fake.csv', 'True.csv']

Shape: (21417, 4)
```

	title	text	subject	date
0	As U.S. budget fight looms, Republicans flip t...	WASHINGTON (Reuters) - The head of a conservat...	politicsNews	December 31, 2017
1	U.S. military to accept transgender recruits o...	WASHINGTON (Reuters) - Transgender people will...	politicsNews	December 29, 2017
2	Senior U.S. Republican senator: 'Let Mr. Muell...	WASHINGTON (Reuters) - The special counsel inv...	politicsNews	December 31, 2017
3	FBI Russia probe helped by Australian diplomat...	WASHINGTON (Reuters) - Trump campaign adviser ...	politicsNews	December 30, 2017
4	Trump wants Postal Service to charge 'much mor...	SEATTLE/WASHINGTON (Reuters) - President Donal...	politicsNews	December 29, 2017

```
tokens = []

for i in df['text']:
    token = word_tokenize(i)
    tokens.append(token)
```

```
w2v = gensim.models.Word2Vec(tokens, min_count=1, vector_size=100, window=5, sg=1)
```

```
print("Cosine similarity between 'provide' and 'program' - Skip Gram :
",w2v.wv.similarity('provide', 'program'))
```

```
Cosine similarity between 'provide' and 'program' - Skip Gram :  0.4427517
```

```
print("words that similar to 'program' - Skip Gram : ",w2v.wv.most_similar('program'))
```

```
words that similar to 'program' - Skip Gram :  [('programme', 0.8306229114532471),
('programs', 0.7761519551277161), ('restraints', 0.6887074112892151), ('bond-buying',
0.68561190366745), ('programmes', 0.6845842003822327), ('modernization', 0.6822008490562439),
('arsenal', 0.6789684295654297), ('accelerates', 0.6762107014656067), ('curbed',
0.6741503477096558), ('plan', 0.6666568517684937)]
```

Tasks

Task 1: Cosine Similarity

Use the Cosine Similarity method to determine how similar the following sentences are.

'This is the first document.',
'This document is the second document.',
'And this is the third one.',
'Is this the first document?'

```python
# Task 1: Cosine Similarity
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = ['This is the first document.',
        'This document is the second document.',
        'And this is the third one.',
        'Is this the first document?']

# Represent the sentences as TF-IDF vectors
tfidf_vectorizer = TfidfVectorizer()
vector_matrix = tfidf_vectorizer.fit_transform(docs)

# Calculate the cosine similarity between every pair of sentences
cosine_similarity_matrix = cosine_similarity(vector_matrix)

labels = ['doc1', 'doc2', 'doc3', 'doc4']

print("Documents:")
for i, d in enumerate(docs, 1):
    print(f"  doc{i}: {d}")
print()

df_cosine = pd.DataFrame(cosine_similarity_matrix, index=labels, columns=labels)
df_cosine.round(3)
```

```
Documents:
  doc1: This is the first document.
  doc2: This document is the second document.
  doc3: And this is the third one.
  doc4: Is this the first document?
```

	doc1	doc2	doc3	doc4
doc1	1.000	0.647	0.308	1.000
doc2	0.647	1.000	0.225	0.647
doc3	0.308	0.225	1.000	0.308
doc4	1.000	0.647	0.308	1.000

Task 2: TF-IDF

Use tf-idf method on the sentences below to determine the important words.

'data science is one of the most important fields of science',
'this is one of the best data science courses',
'data scientists analyze data'

```python
# Task 2: TF-IDF
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

docs2 = ['data science is one of the most important fields of science',
         'this is one of the best data science courses',
         'data scientists analyze data']

tfidf = TfidfVectorizer()
result = tfidf.fit_transform(docs2)

df_tfidf = pd.DataFrame(result.toarray(),
                        columns=tfidf.get_feature_names_out(),
                        index=['doc1', 'doc2', 'doc3'])

print("TF-IDF matrix (words as rows, documents as columns):")
display(df_tfidf.round(3).T)

# The important words in each document are the ones with the highest TF-IDF score
print("\nMost important words in each document:\n")
for name, row in df_tfidf.iterrows():
    top = row.sort_values(ascending=False).head(3)
    print(f"{name}: " + ", ".join(f"{w} ({s:.3f})" for w, s in top.items()))
```

```
TF-IDF matrix (words as rows, documents as columns):
```

	doc1	doc2	doc3
analyze	0.000	0.000	0.543
best	0.000	0.400	0.000
courses	0.000	0.400	0.000
data	0.190	0.236	0.641
fields	0.321	0.000	0.000
important	0.321	0.000	0.000
is	0.244	0.304	0.000
most	0.321	0.000	0.000
of	0.488	0.304	0.000
one	0.244	0.304	0.000
science	0.488	0.304	0.000
scientists	0.000	0.000	0.543
the	0.244	0.304	0.000
this	0.000	0.400	0.000

```
Most important words in each document:

doc1: science (0.488), of (0.488), fields (0.321)
doc2: best (0.400), courses (0.400), this (0.400)
doc3: data (0.641), analyze (0.543), scientists (0.543)
```

Word2vec

Task 3:

Download the Simpsons dataset **(simpsons_script_lines.csv)** and apply the preprocessing procedure.
Use the **'spoken_words'** column.

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"[0-9]", '', text)
    text = re.sub(r"[](,”“.’$-]", '', text)
    return text
```

Create a skip gram Word2Vec model as below.
```
Skip_gram_model = gensim.models.Word2Vec(tokens, min_count = 1, vector_size = 100, window = 5, sg = 1)


```

```python
# Task 3: Load the Simpsons dataset and apply the preprocessing procedure
import pandas as pd, re, os, gensim
from nltk.tokenize import word_tokenize

# Look for the file next to the notebook
candidates = ["simpsons_script_lines.csv",
              os.path.join("data", "simpsons_script_lines.csv"),
              os.path.expanduser("~/Downloads/simpsons_script_lines.csv")]
csv_path = next((p for p in candidates if os.path.exists(p)), None)

if csv_path is None:
    raise FileNotFoundError(
        "simpsons_script_lines.csv was not found. "
        "Place it in the same folder as this notebook and run the cell again.")

simpsons = pd.read_csv(csv_path)
print("File:", csv_path)
print("Shape:", simpsons.shape)
print("Columns:", list(simpsons.columns))

# The 'spoken_words' column contains empty rows, so they are removed first
simpsons = simpsons.dropna(subset=['spoken_words']).reset_index(drop=True)
print("Lines after removing the empty ones:", len(simpsons))


def clean_text(text):
    text = text.lower()                          # lowercasing
    text = re.sub(r"[0-9]", '', text)            # remove digits
    text = re.sub(r"]()(,””.’$-]", '', text)   # remove punctuation
    return text


simpsons['clean'] = simpsons['spoken_words'].apply(clean_text)

print("\nBefore:", simpsons['spoken_words'][0])
print("After :", simpsons['clean'][0])

# Tokenize every line
tokens = [word_tokenize(line) for line in simpsons['clean']]
print("\nNumber of tokenized lines:", len(tokens))
print("Example of a tokenized line:", tokens[0][:15])

# Create the Skip-gram Word2Vec model (sg=1 means Skip-gram)
Skip_gram_model = gensim.models.Word2Vec(tokens, min_count=1,
                                         vector_size=100, window=5, sg=1)

print("\nVocabulary size:", len(Skip_gram_model.wv))
```

```
File: simpsons_script_lines.csv
Shape: (158314, 2)
Columns: ['raw_character_text', 'spoken_words']
Lines after removing the empty ones: 131855

Before: No, actually, it was a little of both. Sometimes when a disease is in all the
magazines and all the news shows, it's only natural that you think you have it.
After : no actually it was a little of both sometimes when a disease is in all the magazines
and all the news shows it's only natural that you think you have it

Number of tokenized lines: 131855
Example of a tokenized line: ['no', 'actually', 'it', 'was', 'a', 'little', 'of', 'both',
'sometimes', 'when', 'a', 'disease', 'is', 'in', 'all']

Vocabulary size: 44282
```

Task 4

Use: wv.most_similar() method to :

1. Find the words similar to “homer”.
2. Find the words similar to “marge”.
3. Find the words similar to “bart”

```
# Task 4: Use wv.most_similar() to find the words similar to each character
for word in ['homer', 'marge', 'bart']:
    print(f"Words similar to '{word}':")
    for w, score in Skip_gram_model.wv.most_similar(word):
        print(f"   {w:<20} {score:.4f}")
    print()
```

```
Words similar to 'homer':
   abe                  0.8745
   marge                0.8550
   ned                  0.8462
   grampa               0.8404
   bartholomew          0.8366
   apu                  0.8236
   eliza                0.8196
   allison              0.8138
   sweetheart           0.8128
   bart                 0.8126

Words similar to 'marge':
   abe                  0.8561
   homer                0.8550
   sweetie              0.8392
   apu                  0.8299
   sweetheart           0.8221
   allison              0.8186
   lisa                 0.8181
   maude                0.8125
   becky                0.8123
   eliza                0.8114

Words similar to 'bart':
   lisa                 0.8857
   milhouse             0.8850
   eliza                0.8408
   grampa               0.8397
   apu                  0.8372
   sweetie              0.8352
   jessica              0.8344
   janey                0.8265
   allison              0.8248
   kirk                 0.8243
```

Task 5

Use the wv.doesnt_match() method to :

1. Find which of 'jimbo', 'milhouse’, and 'kearney’ does not belong to the list.
2. Find the odd one among "nelson", "bart", and "milhouse".
3. Find the odd one among ‘homer', 'patty', and ‘selma'.

Hint: You need to pass the strings as List

```
# Task 5: Use wv.doesnt_match() to find the odd word in each list
groups = [['jimbo', 'milhouse', 'kearney'],
          ['nelson', 'bart', 'milhouse'],
          ['homer', 'patty', 'selma']]

for g in groups:
    odd = Skip_gram_model.wv.doesnt_match(g)
    print(f"{g}  ->  the odd one out is: '{odd}'")
```

```
['jimbo', 'milhouse', 'kearney']  ->  the odd one out is: 'milhouse'
['nelson', 'bart', 'milhouse']  ->  the odd one out is: 'nelson'
['homer', 'patty', 'selma']  ->  the odd one out is: 'homer'
```

Summary of the Tasks

Task 1 — Cosine Similarity. The four sentences were converted into TF-IDF vectors and compared pairwise:

	doc1	doc2	doc3	doc4
doc1	1.000	0.647	0.308	**1.000**
doc2	0.647	1.000	0.225	0.647
doc3	0.308	0.225	1.000	0.308
doc4	**1.000**	0.647	0.308	1.000

The most interesting result is that doc1 ("*This is the first document.*") and doc4 ("*Is this the first document?*") have a similarity of exactly **1.000**, even though one is a statement and the other is a question. This happens because TF-IDF is a bag-of-words representation: it records which words occur and how often, but discards word order, and the tokenizer removes punctuation, so the two sentences end up with identical vectors. doc3 is the least similar to the others because it shares only the common words *and*, *this* and *is*.

Task 2 — TF-IDF. The words with the highest TF-IDF weight in each document are:

Document	Most important words
doc1	science (0.488), of (0.488), fields (0.321)
doc2	best (0.400), courses (0.400), this (0.400)
doc3	data (0.641), analyze (0.543), scientists (0.543)

A word scores highly when it is frequent inside its own document but rare across the rest of the corpus. For example *data* dominates doc3 because it appears twice in a very short sentence, while *science* leads doc1 because it is repeated there. Words such as *analyze* and *scientists* score highly because they appear in only one document, whereas words shared by all three documents receive low weights — which is exactly the purpose of the IDF term.

Task 3 — Pre-processing and the Skip-gram model. The Simpsons dataset contains **158,314** rows with the columns `raw_character_text` and `spoken_words`. The `spoken_words` column has 26,459 empty rows, which were removed first, leaving **131,855** lines of dialogue. Each line was then lower-cased, stripped of digits and punctuation, and tokenized. A Skip-

gram Word2Vec model (`min_count=1, vector_size=100, window=5, sg=1`) was trained on these lines and produced a vocabulary of **44,282** words.

Task 4 — Most similar words.

Word	Top 5 most similar words
homer	abe (0.875), marge (0.855), ned (0.846), grampa (0.840), bartholomew (0.837)
marge	abe (0.856), homer (0.855), sweetie (0.839), apu (0.830), sweetheart (0.822)
bart	lisa (0.886), milhouse (0.885), eliza (0.841), grampa (0.840), apu (0.837)

The model was never told that these are names, yet every neighbour it returns is also a name or a term of address. This is because Word2Vec learns from context: characters are addressed in the same positions in a sentence, so their vectors end up close together. The neighbours also reflect the show itself — *bart* is closest to *lisa* and *milhouse*, the two characters he speaks with most.

Task 5 — Odd one out.

List	Odd one out
jimbo, milhouse, kearney	**milhouse**
nelson, bart, milhouse	**nelson**
homer, patty, selma	**homer**

`doesnt_match()` computes the mean vector of the list and returns the word furthest from it. The three answers match the relationships in the show: *jimbo* and *kearney* are the school bullies, so *milhouse* is the outsider; *bart* and *milhouse* are best friends, so *nelson* stands apart; and *patty* and *selma* are the two sisters, so *homer* is the odd one. This shows that the embeddings captured real social relations purely from the contexts in which the names appear in the dialogue.